

JavaScript / TypeScript / React / Redux / Next.js / Angular

Design Patterns Cheat Sheet

Colorful cheat sheet with medium-sized code snippets.

Module Pattern: Encapsulation & no globals.

```
const counter = (() => { let v = 0; return { inc: () => v++, get: () => v }; })();
```

Factory Pattern: Create objects based on conditions.

```
function createLogger(type) { if (type === 'file') return new FileLogger();  
return new ConsoleLogger(); }
```

Singleton Pattern: Global shared instance.

```
class Store { static inst; static get() { return this.inst ??= new Store();  
} }
```

Observer Pattern: Pub-sub event updates.

```
const bus = new EventTarget();  
bus.addEventListener('u', ()=>console.log('updated')); bus.dispatchEvent(new  
Event('u'));
```

Strategy Pattern: Switch algorithms at runtime.

```
const pay = { card:a=>a*1.02, upi:a=>a };
```

Decorator Pattern: Modify behavior without touching source.

```
function Log(t,k,d){ const f=d.value;  
d.value=(...a)=>{console.log('run');return f(...a)}; }
```

HOC Pattern: Wrap functions/components.

```
const withLog = fn => (...a)=>{ console.log('call'); return fn(...a); };
```